



دانشگاه اصفهان
دانشکده مهندسی کامپیوتر

تمرین سری دوم درس یادگیری تقویتی

استاد درس : دکتر حسین کارشناس

تدریس یارها : عماد آرزومند - علیرضا سرلک

بهار 1405

توضیحات تمرین

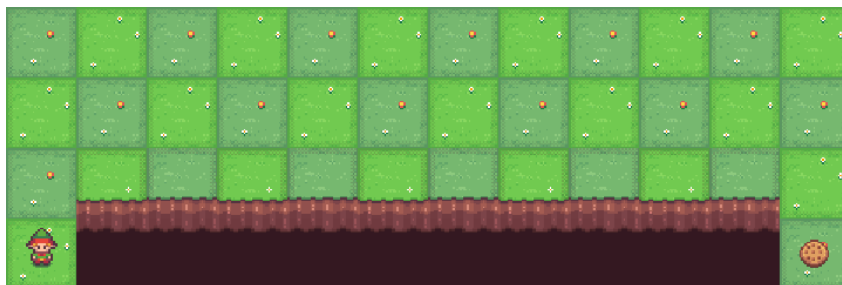
در این تمرین قصد داریم الگوریتم‌های پایه‌ای کنترل و پیش‌بینی مبتنی بر مونت کارلو و تفاوت زمانی (Temporal Difference) را در یک محیط شبکه‌ای تصادفی پیاده‌سازی کرده و رفتار آن‌ها را با یکدیگر مقایسه کنیم. هدف اصلی، درک عمیق‌تر تفاوت بین روش‌های مبتنی بر اپیزود کامل (MC) و روش‌های Bootstrapping، تفاوت سیاست‌های On-policy و Off-policy و همچنین مشاهده چالش‌هایی نظیر سوگیری بیشینه‌سازی (Maximization Bias) در عمل است. شما در این تمرین الگوریتم‌های Q-Learning, SARSA, Double Q-Learning و Expected SARSA را از پیاده‌سازی خواهید کرد.

معیار ارزیابی تمرین گزارش و تحویل حضوری آن است. به دلیل وجود ابزارهای هوش مصنوعی، تسلط شما در زمان تحویل تاثیر بالایی در نمره شما خواهد داشت. انجام تمرین باید به صورت فردی صورت گیرد و مطالعه کتاب Sutton میتواند اطلاعات کاملی برای شما فراهم آورد.

*** در صورت استفاده از ابزارهای هوش مصنوعی، نام ابزار استفاده شده را بیان کنید.**

با تشکر

محیطی که در این تمرین با آن کار خواهید کرد، نسخه تغییر یافته‌ای از مسئله کلاسیک ¹ Cliff Walking است. در این محیط عامل باید مسیری را از نقطه شروع به نقطه هدف پیدا کند، در حالی که لبه‌ی پایینی نقشه یک صخره خطرناک است. برای نزدیک‌تر شدن به شرایط دنیای واقعی، ما عنصر عدم قطعیت را با پارامتری به نام `slip_rate` به محیط اضافه کرده‌ایم.



قوانین و پویایی‌های محیط:

- ابعاد محیط: یک شبکه مستطیلی با ابعاد 12×4 .
- نقطه شروع (S): گوشه پایین سمت چپ.
- نقطه هدف (G): گوشه پایین سمت راست.
- صخره: تمام خانه‌های بین نقطه شروع و هدف در ردیف پایین.
- فضای عمل: عامل در هر وضعیت می‌تواند ۴ حرکت انجام دهد:

بالا (0)

راست (1)

پایین (2)

چپ (3)

- احتمال لغزش: با احتمال $1 - \text{slip_rate}$ عامل در جهت انتخابی حرکت می‌کند و با احتمال `slip_rate` به یکی از جهت‌های عمود بر آن لیز می‌خورد. خروج از نقشه باعث درجا زدن می‌شود. (مثلاً اگر قصد حرکت به بالا را دارد، ممکن است به چپ یا راست لیز بخورد). در صورتی که حرکت عامل (عمدی یا بر اثر لغزش) او را به خارج از نقشه ببرد، در همان جای خود باقی می‌ماند.

¹ https://gymnasium.farama.org/environments/toy_text/cliff_walking/

سیستم پاداش:

برای اینکه عامل یاد بگیرد در سریع‌ترین زمان و با امن‌ترین مسیر به هدف برسد، پاداش‌ها به شکل زیر تنظیم شده‌اند:

- هر گام معمولی: 1-
- افتادن در صخره: 100- (در این حالت، عامل بلافاصله به نقطه شروع برمی‌گردد اما اپیزود تمام نمی‌شود).
- رسیدن به خانه هدف: 0 (و اتمام اپیزود).
- طولانی شدن اپیزود: اگر طول یک اپیزود از ۲۰۰ قدم بیشتر شود، اپیزود با پاداش 50- به پایان می‌رسد.

نحوه ساخت محیط:

دستورالعمل‌های ابتدایی برای استفاده از این محیط به صورت زیر است. محیط را می‌توانید با دستور زیر ایجاد کنید:

```
env = StochasticCliffWalkingEnv(slip_rate=0.1, max_length=200)
```

برای حرکت کردن در محیط، از تابع

```
env.step(a)
```

استفاده کنید، که در آن a اکشنی است که قصد انجام آن را دارید. این تابع به طور خودکار وضعیت محیط را به‌روز می‌کند و همچنین اطلاعات مفیدی درباره انتقال انجام‌شده ارائه می‌دهد:

```
next_state, reward, done, extra = env.step(a)
```

- **next_state**: وضعیت بعدی محیط را نشان می‌دهد.
- **Reward**: پاداش انجام اکشن. قدر وضعیت فعلی است.
- **Done**: مقداری بولی است که مشخص می‌کند آیا اپیزود به پایان رسیده یا نه.
- **Extra**: اطلاعات اضافی است که محیط ممکن است ارائه دهد.

در این محیط، عامل کنترل حرکت یک کاراکتر در یک دنیای شبکه‌ای با ابعاد 4×12 را به عهده دارد. نقطه شروع (S) در گوشه پایین-چپ و نقطه هدف (G) در گوشه پایین-راست قرار دارد. یک ردیف از خانه‌ها بین این دو نقطه به عنوان "پرتگاه" (Cliff) در نظر گرفته شده‌اند.

- حالات (State): حالات با شماره گذاری از 0 تا 47 مشخص می شوند:

$$S = \{0, 1, 2, \dots, 47\}$$

- اکشن ها (Action): عامل در چهار جهت اصلی حرکت کند:

$$A = \{ \text{چپ (left)}, \text{پایین (down)}, \text{راست (right)}, \text{بالا (up)} \}$$

نرخ تخفیف در محیط 0.99 در نظر گرفته شود. $\gamma = 0.99$

در این بخش قصد داریم الگوریتم‌های مبتنی بر مونت کارلو را روی محیط Stochastic Cliff Walking پیاده‌سازی کرده و به تحلیل موازنه‌ها، نحوه اکتشاف و روش‌های ارزیابی Off-policy بپردازیم.

سوال 1) تولید اپیزود:

تابعی به نام Generate_episode ایجاد کنید که سه ورودی دریافت می‌کند:

- سیاست (π): یک تابع احتمال که به ازای هر وضعیت، توزیعی روی اکشن‌ها ارائه دهد:

$$\pi(\mathbb{N} \rightarrow \mathbb{R}^{|A|}: S \rightarrow p(a|S))$$
 - محیط (env)
 - متغیر بولی (render) که اگر مقدارش True باشد، هر مرحله از اپیزود را بصورت متنی نمایش می‌دهد. با استفاده از دستور: `env.render()`
- خروجی این تابع باید سه‌تایی (states, actions, rewards) باشد که به ترتیب نشان‌دهنده وضعیت‌ها، اکشن‌ها و پاداش‌های دریافتی در اپیزود تولید شده با سیاست π است.

سوال 2) الگوریتم First-Visit Monte Carlo را برای سیاست‌های ϵ -soft پیاده‌سازی کنید تا سیاست بهینه را برای محیط بیابید. (برای مطالعه بیشتر در مورد این نوع سیاست به صفحه 100 کتاب ساتن و بارتو مراجعه کنید.)

- الگوریتم را برای دو مقدار مختلف اپسیلون (0.1 و 0.01) اجرا کنید و یک مسیر نمونه از سیاست نهایی هر کدام را نمایش دهید.
- تحلیل: با توجه به ماهیت تصادفی محیط (احتمال لغزش عامل)، توضیح دهید که مقدار ϵ چه تاثیری بر مسیر انتخابی عامل (میزان ریسک‌پذیری و فاصله گرفتن از لبه پرتگاه) دارد و چرا عامل در ϵ بالاتر، محافظه‌کارتر عمل می‌کند؟

سوال 3) برای اطمینان از صحت عملکرد الگوریتم، بخش قبل را (برای $\epsilon = 0.1$) حداقل ۱۰ بار به طور مستقل اجرا کنید. نمودار میانگین بازده (Average Return) در طول اپیزودها را به همراه نوار انحراف معیار (Standard Deviation Band) رسم کنید تا پایداری یادگیری مشخص شود.

- تحلیل: چرا بازده میانگین (Average Return) سیاست ϵ -soft به‌دست آمده، در حین اجرای فاز یادگیری، کمتر از بازده اجرای یک سیاست کاملاً حریصانه و قطعی در همین محیط است؟

سوال 4) می‌خواهیم ارزش (Value) سه وضعیت استراتژیک زیر را با استفاده از روش نمونه‌برداری اهمیت (Importance Sampling) پیش‌بینی کنیم:

1. وضعیت شروع (گوشه پایین چپ- وضعیت 36)

2. وضعیت لبه پرتگاه (دقیقا یک بلوک بالای پرتگاه- وضعیت 29)

3. وضعیت امن (دورترین فاصله از پرتگاه در ردیف بالا- وضعیت 5)

برای این کار، از یک سیاست تصادفی یکنواخت (Uniform Random Policy) به عنوان سیاست رفتار (Behavior Policy) استفاده کرده و ۱۰,۰۰۰ اپیزود تولید کنید. هدف، ارزیابی سیاست حریصانه (Target Policy) به دست آمده از سوال 2 است.

- دو روش Ordinary Importance Sampling (OIS) و Weighted Importance Sampling (WIS) را برای محاسبه ارزش این سه وضعیت پیاده‌سازی کنید. برای آشنایی بیشتر با دو الگوریتم ordinary importance sampling و WIS importance weighted به کتاب Sutton صفحه 104 پاراگراف آخر مراجعه کنید.

- الزام پیاده‌سازی: در این بخش برای بهینه‌سازی حافظه و زمان، از فرمول‌های به‌روزرسانی افزایشی (Incremental Implementation) (مطابق بخش 5.6 کتاب ساتن) استفاده کنید.

- تحلیل: نمودار همگرایی ارزش پیش‌بینی شده برای این سه وضعیت را بر حسب تعداد اپیزودها برای هر دو روش OIS و WIS رسم کنید. واریانس دو روش را مقایسه کرده و به طور خاص توضیح دهید که چرا در «وضعیت لبه پرتگاه»، تفاوت عملکرد و واریانس بین OIS و WIS بسیار شدیدتر و چشمگیرتر از سایر وضعیت‌هاست؟

(راهنمایی: به نسبت اهمیت p و تفاوت اکشن‌های دو سیاست رفتار و هدف در این وضعیت توجه کنید).

بخش دوم – تفاوت MC و TD در پیش بینی

در این بخش، قصد داریم عملکرد الگوریتم‌های پیش‌بینی MC و TD را در محیط StochasticCliffWalkingEnv (با ابعاد ۴ در ۱۲ و نرخ لغزش ۰.۱) که در بخش قبل استفاده کردید، مقایسه کنیم. فرض کنید سیاست رفتار عامل، یک سیاست کاملاً تصادفی یکنواخت است (یعنی $\pi(a|s) = 0.25$ برای تمامی اعمال).

برای مقایسه دقیق‌تر، ابتدا باید ارزش واقعی تمامی وضعیت‌ها (v_π) را به دست آوریم.

گام صفر (محاسبه Ground Truth): با استفاده از الگوریتم ارزیابی سیاست تکراری (Iterative Policy Evaluation) و با دسترسی به مدل محیط (ماتریس احتمالات انتقال P)، مقدار دقیق $v_\pi(s)$ را برای تمامی وضعیت‌ها محاسبه کنید. از این مقادیر به عنوان مرجع برای محاسبه خطای الگوریتم‌های نمونه‌برداری استفاده خواهیم کرد.

سوال 1) مقایسه Every-visit MC و TD(0)

الگوریتم‌های پیش‌بینی Every-visit MC و TD(0) را برای تخمین تابع ارزش $V^\pi(s)$ پیاده‌سازی کنید. برای TD(0) از نرخ یادگیری $\alpha=0.01$ استفاده کنید. هر دو الگوریتم را برای ۵۰,۰۰۰ اپیزود اجرا کنید.

در گزارش:

- الف) نمودار یادگیری: برای ۳ وضعیت استراتژیک انتخاب شده در بخش قبل (وضعیت شروع ۳۶، وضعیت لبه پرتگاه ۲۹، وضعیت امن ۵)، روند همگرایی $V(s)$ را در طول اپیزودها برای هر دو الگوریتم در کنار هم (در ۳ نمودار مجزا برای هر وضعیت) رسم کنید. در این نمودارها، خط چین افقی نشان‌دهنده مقدار واقعی $v_\pi(s)$ (به دست آمده از گام صفر) باشد.
- ب) نمودار خطای RMSE: خطای جذر میانگین مربعات بین تخمین فعلی V و مقدار واقعی v_π (با میانگین‌گیری روی تمامی ۴۸ وضعیت) را در طول اپیزودها برای هر دو الگوریتم محاسبه کرده و در یک نمودار رسم کنید.
- ج) تحلیل: بر اساس نمودارها، تفاوت سرعت همگرایی و واریانس (نوسانات) این دو الگوریتم را تحلیل کنید. چرا در وضعیت ۲۹ (لبه پرتگاه)، رفتار نوسانی MC نسبت به TD(0) تفاوت چشمگیری دارد؟ این موضوع را با توجه به مفهوم Bootstrapping توضیح دهید.

بخش دوم – تفاوت MC و TD در پیش بینی

سوال 2) پیش‌بینی n-step TD و موازنه بایاس-واریانس

الگوریتم پیش‌بینی n-step TD را پیاده‌سازی کنید. این الگوریتم پلی بین TD(0) و MC است.

الگوریتم را با مقادیر $n \in \{1, 2, 4, 8, 100\}$ و $\alpha = 0.01$ به مدت ۵۰,۰۰۰ اپیزود اجرا کنید.

در گزارش

الف) نمودار خطای RMSE (مشابه سوال ۱-ب) را برای تمامی مقادیر n در یک شکل رسم کنید.

ب) تحلیل پارامتر n: کدام مقدار n سریع‌ترین افت خطا (بهترین عملکرد) را دارد؟ با استفاده از مفاهیم بایاس (Bias) و واریانس (Variance)، توضیح دهید که چرا مقادیر خیلی کم یا خیلی زیاد n لزوماً بهترین انتخاب نیستند.

سوال ۳) هم‌ارزی n-step TD و Monte Carlo (بخش امتیازی/تحلیلی عمیق)

انتظار می‌رود که وقتی n بسیار بزرگ است (مثلاً $n = 100$ که از طول اکثر اپیزودها بیشتر است)، الگوریتم n-step TD دقیقاً معادل Every-visit MC عمل کند. اما اگر نمودار $n = 100$ را با MC در سوال ۱ مقایسه کنید، تفاوت خواهید دید.

دلیل این امر، تفاوت در نحوه به‌روزرسانی و نرخ یادگیری است (در MC ضمنی از میانگین‌گیری استفاده می‌شود اما در TD ما از α ثابت استفاده کردیم).

در گزارش:

- الف) اثبات ریاضی: فرمول دقیق و پویایی برای نرخ یادگیری α در الگوریتم n-step TD (وقتی $n \rightarrow \infty$) ارائه دهید تا این الگوریتم از نظر ریاضی و عملکردی دقیقاً معادل Every-visit MC شود.
- ب) اثبات تجربی: کد n-step TD خود را با این α جدید (که تابعی از تعداد بازدید وضعیت‌ها خواهد بود) تغییر دهید. الگوریتم را اجرا کرده و نمودار RMSE آن را روی نمودار Every-visit MC سوال ۱ بیندازید. آیا کاملاً بر هم منطبق شدند؟

در این بخش قصد داریم عملکرد الگوریتم‌های کنترل مبتنی بر تفاوت زمانی (TD) را در محیط تصادفی و پرخطر StochasticCliffWalkingEnv پیاده‌سازی و مقایسه کنیم. برای تمامی قسمت‌های این سوال، پارامترهای محیط را به صورت $\gamma = 0.99$ و $slip_rate = 0.1$ در نظر بگیرید.

سوال 1) مقایسه SARSA و Q-Learning

الگوریتم‌های SARSA (رویکرد On-policy) و Q-learning (رویکرد Off-policy) را پیاده‌سازی کنید. هر دو الگوریتم را برای ۵۰۰۰ اپیزود و روی ۱۰ اجرای مستقل آموزش دهید. ($\epsilon = 0.1$ $\alpha = 0.1$)

- نمودار میانگین بازده (Average Return) در طول اپیزودها را همراه با نوار اطمینان (Confidence Band) برای هر دو روش در یک شکل رسم کنید. همچنین سیاست نهایی یادگرفته‌شده توسط هر دو الگوریتم را استخراج و روی نقشه محیط به صورت بصری نمایش دهید.
- می‌دانیم که در محیط قطعی Cliff Walking، روش Q-learning مسیر لب پرتگاه و SARSA مسیر دور از پرتگاه را یاد می‌گیرند. بررسی کنید که در این نسخه تصادفی ($slip_rate = 0.1$)، آیا مسیر لبه پرتگاه همچنان بهینه واقعی است؟ چرا عملکرد Q-learning در فاز آموزش بدتر از SARSA است اما از نظر تئوری تضمین می‌شود که به ارزش بهینه Q^* همگرا شود؟

سوال 2) کاهش واریانس با Expected SARSA

لگوریتم Expected SARSA را پیاده‌سازی کنید. با همان تنظیمات قسمت قبل، الگوریتم را اجرا کرده و نمودار یادگیری آن را روی همان نمودار سوال ۱ (جهت مقایسه همزمان هر سه الگوریتم) اضافه کنید.

- توضیح دهید که چرا Expected SARSA نسبت به SARSA واریانس کمتری در تخمین‌ها دارد و معمولاً سریع‌تر به پایداری می‌رسد. تاثیر استفاده از نرخ یادگیری بزرگ‌تر (مثلاً $\alpha = 0.5$) را بر روی پایداری این روش در مقایسه با SARSA تحلیل کنید.

سوال 3) Double Q-Learning و Maximum Bias

روش Q-learning به دلیل استفاده از عملگر بیشینه‌سازی (max) در به‌روزرسانی مقدار هدف، به ویژه در محیط‌های تصادفی مستعد پدیده Maximization Bias است.

الگوریتم Double Q-Learning را پیاده‌سازی کنید. نمودار یادگیری Q-learning و Double Q-learning را رسم و مقایسه کنید. همچنین مقدار تخمینی $Q(S_{start}, A_{best})$ را برای هر دو الگوریتم در طول اپیزودها ذخیره کرده و روند تغییرات آن را روی یک نمودار رسم کنید.

- با استناد به نمودارها نشان دهید که چگونه Q-learning ارزش وضعیت شروع را در مراحل اولیه (به دلیل نویز محیط) بیش‌برآورد می‌کند و چگونه Double Q-learning با جداسازی فرآیند انتخاب عمل از ارزیابی عمل، این سوگیری را مهار می‌سازد.

سوال 4) مقایسه جامع کنترل TD و MC

با توجه به نتایج به‌دست‌آمده از بخش اول (MC Control) و این بخش، عملکرد کلی این دو خانواده از الگوریتم‌ها را در محیط StochasticCliffWalkingEnv مقایسه کنید. تحلیل شما باید حداقل شامل موارد زیر باشد:

- نرخ همگرایی (Convergence Rate): کدام دسته از الگوریتم‌ها سریع‌تر به یک سیاست قابل قبول می‌رسند؟ پدیده Bootstrapping در اینجا چه تاثیری دارد؟
- عملکرد نهایی
- واریانس در طول یادگیری: میزان نوسانات در طول یادگیری کدام رویکرد بیشتر است و چرا؟
- نیاز به حافظه و محاسبات: مزایا و معایب به‌روزرسانی آنلاین در طول اپیزود (TD) نسبت به به‌روزرسانی در انتهای اپیزود (MC) را در این محیط خاص بیان کنید.